AI LAB 3

EXPERIMENT 9 - Forward Chaining

AIM: To implement forward chaining using rule-based medical appointment eligibility.

QUESTION:

A healthcare appointment system uses facts and logical rules to determine whether a patient is eligible for a priority consultation. The system derives new facts from existing information by repeatedly applying rules until a conclusion is reached or no further facts can be inferred.

Develop a Python program to implement Forward Chaining using facts and rules.

The program must:

1. Represent the initial facts and inference rules.
2. Check whether the conditions of each rule are satisfied.
3. Infer and add new facts when a rule applies.
4. Continue the inference process until no new facts can be derived.
5. Display the initial facts, newly inferred facts, and final set of facts.

Code:

```
# Experiment 9 - Forward Chaining
# Rule-Based Medical Appointment Eligibility

# Step 1: Define initial facts
facts = {
    "patient_registered",
    "has_fever",
    "high_temperature",
    "severe_pain"
}

# Step 2: Define inference rules
```

```
rules = [
    ({"has_fever", "high_temperature"}, "has_infection"),
    ({"has_infection"}, "needs_medical_attention"),
    ({"severe_pain"}, "needs_priority_care"),
    ({"needs_medical_attention", "needs_priority_care"},
     "eligible_for_priority_consultation")
]

# Step 3: Display initial facts
print("INITIAL FACTS:")
for fact in facts:
    print("-", fact)

# Step 4: Implement Forward Chaining
new_facts = []

print("\nINFERENCE PROCESS:")

while True:
    new_fact_added = False

    for conditions, conclusion in rules:

        # Check whether all conditions are satisfied
        if conditions.issubset(facts) and conclusion not in facts:

            # Infer a new fact
            facts.add(conclusion)
            new_facts.append(conclusion)

            print("\nRule Applied:")
            print("Conditions:", conditions)
            print("New Fact:", conclusion)

            new_fact_added = True

    # Stop when no new facts are derived
    if not new_fact_added:
        break
```

```python
# Step 5: Display newly inferred facts
print("\nNEWLY INFERRED FACTS:")
for fact in new_facts:
    print("-", fact)

# Step 6: Display final facts
print("\nFINAL SET OF FACTS:")
for fact in facts:
    print("-", fact)

# Step 7: Display final conclusion
print("\nFINAL CONCLUSION:")

if "eligible_for_priority_consultation" in facts:
    print("Patient is ELIGIBLE for priority consultation.")
else:
    print("Patient is NOT ELIGIBLE for priority consultation.")
```

Output:

```
INITIAL FACTS:
- patient_registered
- severe_pain
- high_temperature
- has_fever

INFERENCE PROCESS:

Rule Applied:
Conditions: {'has_fever', 'high_temperature'}
New Fact: has_infection

Rule Applied:
Conditions: {'has_infection'}
New Fact: needs_medical_attention

Rule Applied:
Conditions: {'severe_pain'}
New Fact: needs_priority_care

Rule Applied:
Conditions: {'needs_medical_attention', 'needs_priority_care'}
New Fact: eligible_for_priority_consultation
```

```
NEWLY INFERRED FACTS:
- has_infection
- needs_medical_attention
- needs_priority_care
- eligible_for_priority_consultation

FINAL SET OF FACTS:
- has_fever
- has_infection
- needs_medical_attention
- needs_priority_care
- severe_pain
- high_temperature
- patient_registered
- eligible_for_priority_consultation

FINAL CONCLUSION:
Patient is ELIGIBLE for priority consultation.
```

EXPERIMENT 10 - Resolution Refutation

AIM: To verify whether a student qualifies as a happy student using logical statements.

QUESTION:

A university models student well-being using logical facts and rules. It wants to determine whether a student is happy based on the conditions that the student enjoys learning and has sufficient free time.

Develop a Python program to implement Resolution Refutation to prove whether a student is happy.

The program must:

1. Represent the knowledge base using logical statements.
2. Convert the relevant statements into Conjunctive Normal Form (CNF).
3. Negate the goal to be proved and add it to the knowledge base.
4. Apply the resolution rule to derive new clauses.
5. Identify the empty clause, if obtained, to demonstrate a contradiction and establish the goal.

Code:

```
# Experiment 10 - Resolution Refutation

# Step 1: Define Knowledge Base in CNF
# E = Student enjoys learning
# F = Student has sufficient free time
# H = Student is happy

knowledge_base = [
    {"E"},
    {"F"},
    {"~E", "~F", "H"}
]

# Step 2: Display Knowledge Base
print("KNOWLEDGE BASE (CNF):")

for clause in knowledge_base:
    print(clause)

# Step 3: Negate the Goal
# Goal: H
# Negated Goal: ~H

negated_goal = {"~H"}

print("\nNEGATED GOAL:")
print(negated_goal)

# Add negated goal to knowledge base
clauses = knowledge_base + [negated_goal]


# Step 4: Resolution Function
def resolve(c1, c2):

    new_clauses = []

    for literal in c1:
```

```
            complement = (
                literal[1:] if literal.startswith("~")
                else "~" + literal
            )

            if complement in c2:

                resolvent = (c1 - {literal}) | (c2 - {complement})

                # Check for tautology
                if not any(
                    (x[1:] if x.startswith("~") else "~" + x)
                    in resolvent
                    for x in resolvent
                ):
                    new_clauses.append(resolvent)

    return new_clauses


# Step 5: Apply Resolution Refutation
print("\nRESOLUTION PROCESS:")

while True:

    new_clauses = set()
    added = False

    for i in range(len(clauses)):

        for j in range(i + 1, len(clauses)):

            resolvents = resolve(clauses[i], clauses[j])

            for clause in resolvents:

                # Convert set into frozenset
                frozen_clause = frozenset(clause)
```

```
                if frozen_clause not in new_clauses and clause not in
clauses:

                    print("\nResolving:", clauses[i], "AND", clauses[j])
                    print("New Clause:", clause)

                    new_clauses.add(frozen_clause)
                    added = True

                    # Check for empty clause
                    if len(clause) == 0:

                        print("\nEmpty clause obtained!")
                        print("Contradiction found.")
                        print("Therefore, the student is HAPPY.")

                        added = False
                        break

            if any(len(c) == 0 for c in new_clauses):
                break

        if any(len(c) == 0 for c in new_clauses):
            break

    # Check whether empty clause was obtained
    if any(len(c) == 0 for c in new_clauses):
        break

    if not added:
        print("\nEmpty clause not obtained.")
        print("Unable to prove the goal.")
        break

    # Add new clauses to knowledge base
    clauses.extend([set(c) for c in new_clauses])
```

Output:

```
KNOWLEDGE BASE (CNF):
{'E'}
{'F'}
```

```
{'H', '~E', '~F'}

NEGATED GOAL:
{'~H'}

RESOLUTION PROCESS:

Resolving: {'E'} AND {'H', '~E', '~F'}
New Clause: {'H', '~F'}

Resolving: {'F'} AND {'H', '~E', '~F'}
New Clause: {'H', '~E'}

Resolving: {'H', '~E', '~F'} AND {'~H'}
New Clause: {'~E', '~F'}

Resolving: {'E'} AND {'H', '~E'}
New Clause: {'H'}

Resolving: {'E'} AND {'~E', '~F'}
New Clause: {'~F'}

Resolving: {'F'} AND {'~E', '~F'}
New Clause: {'~E'}

Resolving: {'E'} AND {'~E'}
New Clause: set()

Empty clause obtained!
Contradiction found.

Therefore, the student is HAPPY.
```

EXPERIMENT 11: Family Tree

AIM: To represent family relationships in an expert system.

QUESTION:

Develop a program to represent family relationships using facts and rules. Given a family tree, identify the parents, children, and grandparents of specified individuals.

Code:

```
# Experiment 11 - Family Tree

parents = [
    ("Ravi", "Arun"),
    ("Ravi", "Meena"),
    ("Arun", "Kiran"),
    ("Meena", "Priya"),
    ("Suresh", "Ravi")
]


def is_parent(parent, child):
    return (parent, child) in parents


def get_children(parent):
    return [child for p, child in parents if p == parent]


def get_parents(child):
    return [parent for parent, c in parents if c == child]


def get_grandparents(person):
    grandparents = []

    for parent in get_parents(person):
        grandparents.extend(get_parents(parent))

    return grandparents
```

```
# Display family relationships
person = "Kiran"

print("Family Tree Relationships")
print("-------------------------")

print("Parents of", person, ":", get_parents(person))

print("Children of Ravi:", get_children("Ravi"))

print("Grandparents of", person, ":", get_grandparents(person))

print("\nIs Ravi a parent of Arun?", is_parent("Ravi", "Arun"))

print("Is Suresh a grandparent of Kiran?",
      "Suresh" in get_grandparents("Kiran"))
```

Output:

```
Family Tree Relationships
-------------------------
Parents of Kiran : ['Arun']
Children of Ravi: ['Arun', 'Meena']
Grandparents of Kiran : ['Ravi']

Is Ravi a parent of Arun? True
Is Suresh a grandparent of Kiran? False
```

EXPERIMENT 12: Pattern Matching in a Healthcare Symptom Assessment System

AIM: To implement rule-based healthcare support for preliminary symptom assessment.

QUESTION:

A healthcare support system receives a patient's description of symptoms in natural language. The system must identify relevant symptom keywords and phrases, match them against predefined rules, and return suitable guidance or recommend professional assessment.

Develop a Python program that applies pattern matching to patient-reported symptoms and identifies predefined symptom patterns to generate relevant guidance.

Code

```
import re

# Healthcare symptom patterns

symptom_patterns = {

    "Emergency Warning Signs": [
        r"\bchest pain\b",
        r"\bdifficulty breathing\b",
        r"\bshortness of breath\b",
        r"\bface drooping\b",
        r"\bsudden weakness\b",
        r"\bsevere bleeding\b"
    ],

    "Respiratory Symptoms": [
        r"\bcough\b",
        r"\bsore throat\b",
        r"\brunny nose\b",
        r"\bfever\b"
    ],

    "Digestive Symptoms": [
        r"\bnausea\b",
        r"\bvomiting\b",
        r"\bstomach pain\b",
        r"\babdominal pain\b",
        r"\bdiarrhea\b"
    ],

    "Neurological or General Symptoms": [
        r"\bheadache\b",
        r"\bdizziness\b",
        r"\bfatigue\b",
        r"\bweakness\b"
    ]

}
```

```
# Guidance associated with matched patterns

guidance = {

    "Emergency Warning Signs":
        "Seek emergency medical care immediately. Do not delay care "
        "while using this system.",

    "Respiratory Symptoms":
        "Consider professional medical advice if symptoms persist, "
        "worsen, or are accompanied by breathing difficulty.",

    "Digestive Symptoms":
        "Maintain hydration if appropriate and seek medical advice "
        "for persistent or worsening symptoms.",

    "Neurological or General Symptoms":
        "Consider medical assessment if symptoms persist or worsen. "
        "Sudden weakness or other sudden neurological changes require "
        "emergency assessment."

}


def match_symptoms(patient_input):

    text = patient_input.lower()

    matched_categories = []
    matched_symptoms = []

    for category, patterns in symptom_patterns.items():

        category_matches = []

        for pattern in patterns:

            if re.search(pattern, text):
```

```
                category_matches.append(
                    re.search(pattern, text).group()
                )

        if category_matches:

            matched_categories.append(category)
            matched_symptoms.extend(category_matches)

    return matched_categories, matched_symptoms


def healthcare_assistant():

    print("HEALTHCARE SYMPTOM ASSESSMENT SYSTEM")
    print("-" * 40)

    print("Describe the symptoms in one sentence.")
    print("Example: I have a fever, cough and sore throat.")

    patient_input = input("\nPatient: ").strip()

    if not patient_input:

        print("System: Please enter a symptom description.")
        return

    categories, symptoms = match_symptoms(patient_input)

    print("\nASSESSMENT REPORT")
    print("-" * 40)

    if not categories:

        print("Matched symptoms: None")

        print("System: No predefined pattern was matched.")

        print("Please describe the symptoms more clearly or "
              "consult a healthcare professional.")
```

```
        return

    print("Matched symptoms:", ", ".join(sorted(set(symptoms))))

    print("Matched categories:", ", ".join(categories))

    # Emergency patterns take priority over other matches

    if "Emergency Warning Signs" in categories:

        print("\nPriority: EMERGENCY WARNING SIGN")

        print("Guidance:", guidance["Emergency Warning Signs"])

    else:

        print("\nPriority: NON-EMERGENCY PATTERN MATCH")

        for category in categories:

            print(f"{category}: {guidance[category]}")

    print("\nNote: This system does not diagnose diseases "
          "or replace professional medical assessment.")


healthcare_assistant()
```

Output:

```
HEALTHCARE SYMPTOM ASSESSMENT SYSTEM
----------------------------------------
Describe the symptoms in one sentence.
Example: I have a fever, cough and sore throat.

Patient: fever

ASSESSMENT REPORT
----------------------------------------
Matched symptoms: fever
Matched categories: Respiratory Symptoms

Priority: NON-EMERGENCY PATTERN MATCH
```

Respiratory Symptoms: Consider professional medical advice if symptoms persist, worsen, or are accompanied by breathing difficulty.

Note: This system does not diagnose diseases or replace professional medical assessment.